

Networking Fundamentals

Why this chapter matters: every distributed system is just programs talking over a network. Latency, timeouts, retries, load balancers, CDNs, DNS failures, “connection refused” — all of it is networking. If you understand what happens to one request from your laptop to a server, you can reason about almost any scaling or reliability problem.

🔗 QUICK RECALL — Chapter map

- 3.1 The problem & layered models → 3.2 Encapsulation → 3.3 MAC, ARP, switches → 3.4 IP, CIDR, routing → 3.5 NAT
- 3.6 Ports, sockets, 5-tuple → 3.7 TCP → 3.8 UDP & QUIC → 3.9 DNS → 3.10 HTTP, TLS & “what happens when you type a URL”
- 3.11 SSH, VPN, proxies → 3.12 Latency, bandwidth, throughput, BDP → 3.13 Network devices & firewalls
- 3.14 Troubleshooting toolbox & error meanings → 3.15 Design implications → 3.16 Practice, Q&A, cheat sheet

💡 SENIOR INSIGHT — Builds on Chapters 1-2

- Ch. 1: sockets are file descriptors watched by epoll — that is how one Node thread serves thousands of connections. Ch. 2: a data-center round trip is ~0.5 ms and a cross-continent one ~150 ms. This chapter explains *why* and what each round trip is spent on.

3.1 What problem does networking solve?

Your laptop wants to talk to a Google server across the Internet. It must answer five separate questions, and each answer lives in a different layer:

#	Question	Answer (identifier / protocol)	Layer
1	Which physical device on my local link receives this frame?	MAC address (+ ARP, Ethernet/Wi-Fi)	2 Data Link
2	Which network / host in the world should the packet go to?	IP address (+ routing)	3 Network
3	Which application on that host?	Port number	4 Transport
4	How should data be transported (reliable, ordered or fast)?	TCP / UDP (or QUIC)	4 Transport
5	What does the message mean?	HTTP, DNS, SSH, SMTP, TLS...	5-7 Application

Two reference models describe the layers:

OSI model (7 layers, conceptual)	TCP/IP model (4 layers, what the Internet uses)	PDU (data unit)	Examples	Device
7 Application 6 Presentation 5 Session	Application	Data / message	HTTP, HTTPS, DNS, SSH, SMTP, TLS, gRPC	Proxy, L7 load balancer, API gateway
4 Transport	Transport	Segment (TCP) / Datagram (UDP)	TCP, UDP, QUIC; ports	Firewall, L4 load balancer
3 Network	Internet	Packet	IPv4, IPv6, ICMP, routing	Router
2 Data Link	Link	Frame	Ethernet, Wi-Fi, ARP; MAC addresses	Switch, access point
1 Physical	(Link)	Bits	Cables, fibre, radio	Hub, repeater, cables

★ **MUST REMEMBER** — OSI mnemonic (bottom → top)

- Please **Do Not Throw Sausage Pizza Away** = Physical, Data link, Network, Transport, Session, Presentation, Application.
- In practice people say “L4 load balancer” (TCP/UDP level) and “L7 load balancer” (HTTP level). Sessions and presentation (TLS, encoding) are folded into the application layer in the real Internet.

3.2 Encapsulation — how data travels down the stack

Each layer wraps the data from the layer above with its own header (and the link layer adds a trailer). The receiver unwraps in reverse order.

Application	HTTP request: GET /index.html ...	
Transport	[TCP header HTTP data]	= segment (ports, seq no.)
Network	[IP header TCP header HTTP data]	= packet (src/dst IP)
Data link	[Ethernet hdr IP TCP data FCS]	= frame (src/dst MAC)
Physical	0101 1100 1010 ...	= bits on wire / radio

◆ **SENIOR INSIGHT** — The key routing insight: IP is end-to-end, MAC is hop-by-hop

- Between your laptop and a server 10 routers away, the **source and destination IP stay the same** (except where NAT rewrites them), but the **MAC addresses are rewritten at every hop** (each router puts its own MAC as source and the next hop’s MAC as destination).
- MTU: a link’s largest frame payload, typically **1500 bytes** on Ethernet. Larger packets are fragmented or dropped (Path MTU Discovery). VPN/tunnel overhead can silently shrink the MTU and cause “works for small requests, hangs for large ones” bugs.

3.3 MAC address, ARP & switches (Layer 2)

A **MAC address** (Media Access Control) is a 48-bit hardware identifier of a network interface, written like 3C:22:FB:9A:01:7E. The first 24 bits identify the vendor (OUI). It is used only for delivery **within one local network (LAN)**.

- MAC answers “**which interface on this local link?**”; IP answers “**which destination network/host?**”.
- **ARP** (Address Resolution Protocol): the host broadcasts “who has 192.168.1.1?”; the owner replies with its MAC; the result is cached in the ARP table.
- A **switch** learns which MAC is behind which port and forwards frames only to that port (a hub repeats to everyone). Traffic within a **broadcast domain** (a LAN/VLAN) never needs a router.
- To reach an IP outside the local subnet, the host sends the frame to the **default gateway’s** MAC (the router), keeping the final IP as destination.

🔧 **CORRECTION TO YOUR ORIGINAL NOTE** — “MAC is globally unique and burned in”

- Mostly true for factory addresses, but MACs can be **changed/spoofed**, and modern phones and laptops use **randomised MAC addresses** on Wi-Fi for privacy. Never use a MAC as a security identity. VMs and containers get virtual MACs.

3.4 IP address, subnets (CIDR) & routing (Layer 3)

An **IP address** is a logical, hierarchical address used to locate and route packets across interconnected networks. **IPv4** is 32 bits (≈ 4.3 billion addresses, exhausted), written as four bytes 192.168.1.20. **IPv6** is 128 bits ($\approx 3.4 \times 10^{38}$ addresses), written in hex 2001:db8::1.

Public vs private addresses

Range	Name	Notes
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private (RFC 1918)	Not routable on the public Internet; reused in every home/company/VPC; need NAT to reach the Internet
100.64.0.0/10	Carrier-grade NAT	ISPs place many customers behind one public IP
127.0.0.0/8 (127.0.0.1, ::1)	Loopback / localhost	Traffic never leaves the machine
169.254.0.0/16	Link-local	Self-assigned when DHCP fails; also cloud metadata (169.254.169.254)
Everything else	Public	Globally routable, allocated by registries

CIDR notation: 192.168.1.0/24 means the first 24 bits identify the network, the remaining 8 bits identify hosts. Addresses in a block = $2^{(32 - \text{prefix})}$. The first address is the network id, the last is the broadcast; so usable hosts = $2^n - 2$ (cloud providers reserve a few more; AWS reserves 5 per subnet).

CIDR	Subnet mask	Total addresses	Typical use
/8	255.0.0.0	16,777,216	Huge private block (10.0.0.0/8)
/16	255.255.0.0	65,536	A whole VPC / large site
/20	255.255.240.0	4,096	Large subnet
/24	255.255.255.0	256 (254 usable)	Standard subnet
/28	255.255.255.240	16 (14 usable)	Tiny subnet
/32	255.255.255.255	1	A single host (firewall rules)

Example: host 192.168.1.20/24 → network 192.168.1.0, hosts .1–.254, broadcast 192.168.1.255. A destination inside this range is reached directly via ARP; anything else is sent to the **default gateway**.

- **Routing:** each router consults a routing table, picks the **longest matching prefix**, and forwards to the next hop. Protocols like BGP (between ISPs) and OSPF (inside a network) build these tables.
- **TTL** (hop limit) is decremented at each router and the packet is dropped at 0 → prevents routing loops. traceroute exploits this.
- **ICMP** carries control messages (echo/ping, “destination unreachable”, “TTL exceeded”). Blocking all ICMP breaks Path-MTU discovery.

◆ SENIOR INSIGHT — Cloud networking layout you will draw in interviews

- VPC 10.0.0.0/16 → **public subnets** (route to an Internet Gateway: load balancers, bastions) and **private subnets** (no inbound Internet; outbound via a **NAT gateway**: app servers, databases). Spread subnets across **availability zones**.
- Plan CIDR blocks so they do **not overlap** between VPCs/on-prem — overlapping ranges make peering and VPN impossible later.

3.5 NAT — Network Address Translation

Private addresses cannot be routed on the Internet, and public IPv4 addresses are scarce. **NAT** lets many private devices share one public IP: the router **rewrites the source IP (and port)** of outbound packets and remembers the mapping to translate replies back.

```
Laptop 192.168.1.20:52000 ---> [ Router / NAT ] ---> Server 142.250.x.x:443
                               src rewritten to
                               203.0.113.7:40001      (public IP : chosen port)
```

```
NAT table: 203.0.113.7:40001 <-> 192.168.1.20:52000 (TCP)
Reply to 203.0.113.7:40001 is translated back to 192.168.1.20:52000
```

Type	What it does	Example
SNAT / PAT (masquerade)	Outbound: rewrite source IP+port; many hosts → one public IP	Home router, cloud NAT gateway
DNAT / port forwarding	Inbound: rewrite destination to an internal host/port	Router forwarding :443 to a server; Kubernetes NodePort, load balancer

★ **MUST REMEMBER** — Private IP ≠ “no Internet”

- A private IP is not Internet-routable by itself, but a device can still reach the Internet through NAT: **Private IP → NAT/router → Public IP → Internet**. Inbound connections from outside cannot reach it unless you add port forwarding or a public-facing load balancer.

◆ **SENIOR INSIGHT** — NAT consequences in system design

- **Not a firewall**: it blocks unsolicited inbound as a side effect, but you still need explicit security rules.
- **Port exhaustion**: one public IP has only ~64k ports per destination. A service making huge numbers of outbound calls through one NAT gateway to one destination can hit `ErrorPortAllocation/SNAT exhaustion` → use connection pooling/keep-alive or more NAT IPs.
- Many users behind one NAT share one public IP: IP-based rate limiting or blocking can punish an entire office/college. IPv6 largely removes the need for NAT.

3.6 Ports, sockets & the 5-tuple (Layer 4 addressing)

An IP address finds the host; a **port** finds the **application/process** on it. A TCP/UDP port is a **16-bit number: 0-65535** ($2^{16} = 65,536$ values; port 0 is reserved).

Range	Name	Use
0 – 1023	Well-known / system ports	Standard services; binding needs privileges on Unix
1024 – 49151	Registered ports	Databases, apps (3306, 5432, 8080...)
49152 – 65535	Dynamic / ephemeral (IANA)	Temporary client-side source ports. <i>Linux default: 32768-60999 (~28k ports)</i>

Common ports to memorise

Service	Port	Service	Port	Service	Port
HTTP	80	PostgreSQL	5432	Kafka	9092
HTTPS	443	MySQL	3306	Elasticsearch	9200
SSH	22	MongoDB	27017	RabbitMQ (AMQP)	5672
DNS	53 (UDP/TCP)	Redis	6379	etcd	2379
SMTP	25 / 587	Memcached	11211	NTP	123 (UDP)
FTP	21	RDP	3389	DHCP	67/68 (UDP)

A **socket** is the OS-managed endpoint of communication, identified by **IP:port** (e.g., `10.0.0.5:443`). In Unix it is a file descriptor — which is why Node/Nginx watch sockets with `epoll` (Chapter 1). A socket is more than a string: it carries OS state (buffers, TCP state machine).

A TCP connection is uniquely identified by the **5-tuple**:

Source IP	Source port	Destination IP	Destination port	Protocol
192.168.1.20	52000 (ephemeral)	142.250.x.x	443	TCP

◆ **SENIOR INSIGHT** — How one server port handles 100,000 clients; where limits really come from

- A **listening socket** on port 443 accepts connections; each accepted connection is a new **connected socket** with a different 5-tuple. The server uses just ONE port for all clients.
- Limits: (1) **file descriptors** (`ulimit -n`) — raise it; (2) memory per connection; (3) a **client** (or proxy/load balancer) connecting to one destination IP:port is limited by its ephemeral ports (~28k-64k) — the classic cause of “cannot assign requested address”; (4) **TIME_WAIT** sockets (see TCP).
- Load balancers terminate client connections and open separate connections to backends, so each hop has its own 5-tuple.

3.7 TCP — Transmission Control Protocol

TCP gives applications a **reliable, ordered, error-checked byte stream** on top of unreliable IP packets. It is connection-oriented: both sides keep state about the connection.

What TCP guarantees, and how:

Guarantee	Mechanism
Reliable delivery	Sequence numbers + acknowledgements (ACKs); unacknowledged data is retransmitted after a timeout (RTO) or after 3 duplicate ACKs (fast retransmit)
Ordered	Receiver reorders segments by sequence number before giving bytes to the application
Error-checked	Checksum in every segment; corrupted segments are dropped and resent
Flow control	Receiver advertises its free buffer space (receive window, rwnd) so a fast sender cannot overwhelm a slow receiver
Congestion control	Sender maintains a congestion window (cwnd) : slow start (doubles each RTT) → congestion avoidance (+1 per RTT) → on loss, cut the window (AIMD). Algorithms: Reno, CUBIC, BBR. Sending window = $\min(\text{cwnd}, \text{rwnd})$

The three-way handshake

Client	Server
---- SYN (seq = x) ----->	server in LISTEN state
<--- SYN + ACK (seq = y, ack = x+1) ----	
---- ACK (ack = y+1) ----->	
ESTABLISHED (both sides)	
==== data can now flow =====	

Step	Client says	Server says / meaning
1. SYN	“I want to open a connection; my initial sequence number is x.”	Server learns client can send
2. SYN-ACK	—	“Received; here is my initial sequence number y; I acknowledge x.” Server proves it can receive and send
3. ACK	“Received your y.” Client proves it can receive	Both sides have synchronised sequence numbers → data can flow

★ MUST REMEMBER

- The handshake costs **1 RTT before the first byte of data**. At 150 ms RTT that is 150 ms of pure waiting — the reason for **connection reuse (keep-alive, pooling)** everywhere.
- Closing: four-way **FIN/ACK** exchange. The side that closes first enters **TIME_WAIT** (~60 s on Linux) to absorb stray late packets; thousands of short-lived connections can pile up TIME_WAIT sockets.

◆ SENIOR INSIGHT — TCP details that show up in real incidents

- **Head-of-line blocking**: one lost segment stalls all later bytes on that connection until retransmitted (hurts HTTP/2 multiplexing over one TCP connection → motivation for QUIC).
- **Slow start**: initial cwnd is ~10 segments (~14 KB), so new connections start slow; short flows rarely reach full bandwidth. Another reason to reuse warm connections.
- **SYN flood**: attacker sends SYNs without finishing handshakes to fill the backlog queue; mitigated by SYN cookies, rate limiting, CDN/DDoS shields.
- **Nagle's algorithm + delayed ACK** can add ~40 ms latency to small writes; latency-sensitive apps set TCP_NODELAY.
- **Keep-alive** probes detect dead peers; idle timeouts in load balancers/NAT (often 60 s–350 s) silently drop idle connections → app sees ECONNRESET; configure app-level keep-alive below the idle timeout.

3.8 UDP & QUIC

UDP (User Datagram Protocol) is connectionless and minimal: it just adds ports and a checksum to IP. No handshake, no ordering, no retransmission, no congestion control. Datagrams may be lost, duplicated or arrive out of order — the application decides what to do.

Feature	TCP	UDP
Connection-oriented / 3-way handshake	Yes	No
Reliable delivery, retransmission	Yes (at TCP layer)	No
Ordered byte stream	Yes	No — independent datagrams
Flow control / congestion control	Yes / Yes	No TCP-style mechanism / not inherently
Data model	Byte stream (no message boundaries)	Datagrams (message boundaries kept)
Overhead / header size	Higher (20+ bytes)	Lower (8 bytes)
Broadcast / multicast	No (point to point)	Yes
Typical uses	Web, APIs, databases, SSH, email, file transfer	DNS, DHCP, VoIP, video calls, live streaming, online games, QUIC/HTTP/3, metrics (StatsD)

Choose TCP when all data must arrive intact and in order. **Choose UDP when** you need the lowest latency, late data is worse than lost data (a late voice packet is useless), you want your own error correction, or you need broadcast/multicast (e.g., DHCP — the client has no IP yet, so TCP cannot be used).

◆ SENIOR INSIGHT — QUIC / HTTP/3 — “reliable transport built on UDP”

- QUIC runs over UDP in user space and implements its own reliability, congestion control and **TLS 1.3**. Benefits: connection + encryption setup in **1 RTT** (0-RTT on resumption), **independent streams** (a lost packet blocks only its stream — no TCP head-of-line blocking), and **connection migration** (survives Wi-Fi → mobile switch via connection ID).
- Used by HTTP/3 (Google, Cloudflare, Meta). Some networks block or throttle UDP, so clients fall back to TCP.

3.9 DNS — the Internet's phonebook

The **Domain Name System** translates human-friendly names (example.com) into IP addresses. It is a **hierarchical, distributed, heavily cached** database. It usually runs over **UDP port 53** (TCP for large responses/zone transfers; DoH/DoT encrypt it).

Resolution walk (when nothing is cached)

```
Browser cache -> OS cache (+ /etc/hosts) -> Recursive resolver (ISP, 8.8.8.8, 1.1.1.1)
                                         | cache miss, resolver asks in turn:
1) ROOT servers       : "who handles .com?"           -> addresses of .com TLD servers
2) .com TLD servers   : "who handles example.com?"    -> NS records of the domain
3) AUTHORITATIVE servers : "A record of example.com?" -> 93.184.x.x (TTL 3600)
Resolver caches the answer for TTL seconds and returns it to the client.
```

- **Recursive resolver** does the legwork for clients and caches; **authoritative servers** hold the real records for a zone; root and TLD servers only point onward.
- **Caching at every level** (browser, OS, resolver) is why DNS is fast — and why changes are not instant.

DNS record types

Record	Maps	Example / note
A	Name → IPv4	example.com → 93.184.216.34
AAAA	Name → IPv6	example.com → 2606:2800:...
CNAME	Name → another name (alias)	www → example.com. Cannot coexist with other records at the same name, so not allowed at the zone apex (use ALIAS/ANAME features)
NS	Zone → its name servers (delegation)	example.com → ns1.dnsprovider.com
MX	Domain → mail servers (with priority)	10 mail.example.com
TXT	Free text	SPF, DKIM, domain verification
SOA / PTR / SRV	Zone metadata / reverse lookup (IP → name) / service location	SRV used by service discovery

★ MUST REMEMBER — TTL — the DNS trade-off

- **TTL (Time To Live)** = seconds a record may be cached. **High TTL** (hours): fewer lookups, faster, more resilient to DNS outages, but changes propagate slowly. **Low TTL** (30–60 s): fast failover and blue-green cutovers, but more DNS traffic and latency.
- Migration tip: **lower the TTL a day before** moving servers, switch the record, then raise TTL again. "DNS propagation" is really just caches expiring.

Tools (note: your original had the commands run together; these are the separate commands):

```
nslookup example.com           # simple lookup
nslookup -type=NS example.com  # name servers of the domain
dig example.com                # full answer with TTL, server used, query time
dig example.com A +short       # just the IPs
dig NS example.com             # NS records
dig @8.8.8.8 example.com       # ask a specific resolver
dig +trace example.com         # walk root -> TLD -> authoritative like a resolver
```

◆ SENIOR INSIGHT — DNS in system design

- **DNS-based load balancing / routing:** return multiple A records (round robin), weighted, latency-based or geo-based answers (Route 53, Cloudflare) — covered in Chapter 6. Limit: clients cache answers, so DNS cannot remove a dead server instantly; health-checked load balancers do that.
- **DNS is a single point of failure for everything:** use 2+ independent providers/NS records for critical domains; DNS outages (and DDoS on DNS providers) have taken down major sites.
- **Service discovery** inside clusters uses DNS (Kubernetes CoreDNS: `svc.namespace.svc.cluster.local`). In Node, `dns.lookup` uses the `libuv` thread pool — slow DNS can starve it (Chapter 1).

3.10 HTTP, TLS & “what happens when you type a URL”

HTTP is the request/response application protocol of the web (verbs, status codes, headers and API styles get their own Chapter 4). Here we focus on how it rides on the lower layers.

Version	Transport	Key ideas	Limitation
HTTP/1.1	TCP	Text protocol, persistent connections (keep-alive), one request at a time per connection	Head-of-line blocking at HTTP level → browsers open ~6 connections per host
HTTP/2	TCP + TLS	Binary framing, multiplexing many streams on one connection, header compression (HPACK), server push (deprecated)	TCP-level head-of-line blocking on packet loss
HTTP/3	QUIC (UDP) + TLS 1.3	Independent streams, faster setup, connection migration	UDP may be blocked; more CPU in user space

TLS (Transport Layer Security; “HTTPS” = HTTP over TLS, port 443) provides encryption, integrity and server authentication:

- 1 Client sends supported versions/ciphers + **SNI** (the hostname wanted, so one IP can host many certificates).
- 2 Server replies with its **certificate** (public key signed by a Certificate Authority) — the client verifies the chain of trust, expiry and hostname.
- 3 Both sides agree a shared secret using **asymmetric key exchange** (ECDHE → forward secrecy).
- 4 All further data is encrypted with fast **symmetric keys** (e.g., AES-GCM / ChaCha20). TLS 1.3 completes this in 1 RTT (TLS 1.2: 2 RTT).

What happens when you type `https://www.example.com` in the browser? (classic interview question)

#	Step	Round trips	Notes
1	URL parsed; browser/HSTS/cache checks	0	May already have the page cached
2	DNS lookup (browser → OS → resolver → root/TLD/authoritative)	~1 RTT to resolver (0 if cached)	UDP 53; answer cached for TTL
3	TCP three-way handshake with server (ARP/gateway/NAT/routing along the way)	1 RTT	Via CDN edge / load balancer IP
4	TLS handshake, certificate verification	1 RTT (TLS 1.3) / 2 RTT (1.2)	HTTP/3: steps 3+4 combined in 1 RTT
5	HTTP request sent (GET /), reaches load balancer → web server → app → cache/DB	1 RTT + server time	TTFB = time to first byte
6	Server returns HTTP response (status, headers, HTML)	Transfer time	Gzip/Brotli, CDN, cache headers

#	Step	Round trips	Notes
7	Browser parses HTML, discovers CSS/JS/images → repeats 2-6 for those (reusing connections), builds DOM/CSSOM, renders	—	HTTP/2 multiplexing; CDN for static files

★ **MUST REMEMBER** — Cold-start latency budget

- DNS + TCP + TLS1.3 + request = about **4 RTTs before the first response byte**. At 100 ms RTT $\approx$ **400 ms**; with TLS 1.2 $\approx$ 500 ms. Warm connection reuse removes steps 2-4 and leaves $\sim$ 1 RTT.
- This is the whole case for CDNs (shrink RTT by serving from a nearby edge), keep-alive, HTTP/2-3, DNS prefetch/caching and TLS session resumption.

3.11 SSH, VPN & proxies

Tool	What it is	Key facts
SSH	Secure Shell: encrypted remote login, command execution, file transfer (scp/sftp) and tunnelling	Port 22. Prefer key-based auth over passwords; disable root login; use a bastion/jump host or SSM instead of exposing servers. Port forwarding: <code>ssh -L 5433:db.internal:5432 bastion</code> lets you reach a private DB through a tunnel
VPN	Encrypted tunnel over a public network that makes remote machines behave as if on a private network	Remote-access (laptop → company/VPC) or site-to-site (office ↔ cloud). Protocols: IPsec, OpenVPN, WireGuard. Adds latency, reduces MTU. Example: Laptop = encrypted tunnel = Company/VPC network
Forward proxy	Sits in front of <i>clients</i> , makes requests on their behalf	Filtering, caching, anonymity, corporate egress control
Reverse proxy	Sits in front of <i>servers</i> ; clients talk only to it	TLS termination, caching, compression, load balancing (Nginx, HAProxy, Envoy) — Chapter 7

3.12 Latency, bandwidth, throughput & the bandwidth-delay product

These three words are constantly confused. Think of a pipe: **latency** = how long one drop takes to travel through it; **bandwidth** = the pipe's width; **throughput** = how much actually flows.

Term	Meaning	Example
Latency	Delay for data to travel sender → receiver (one-way), or RTT = round trip	API latency = 50 ms
Bandwidth	Maximum theoretical capacity of the link	Network bandwidth = 1 Gbps
Throughput	Actual volume of data (or requests) delivered per unit time	Service throughput = 10,000 req/s
Jitter / Loss	Variation in latency / percentage of lost packets	Voice calls need low jitter; TCP throughput collapses with loss on long links

Components of latency: **propagation** (distance ÷ signal speed; light in fibre $\approx$ 200,000 km/s $\approx$ **5 μ s per km**), **transmission** (size ÷ bandwidth), **queuing** (waiting in router buffers — grows under congestion), and **processing** (routers, servers). You can buy bandwidth; you cannot buy a faster speed of light — Delhi ↔ California is at least $\sim$ 120 ms RTT.

◆ SENIOR INSIGHT — Bandwidth-Delay Product (BDP) — why “a fast link” can still be slow

- **BDP = bandwidth × RTT** = bytes that must be “in flight” to fill the pipe. 1 Gbps × 100 ms = 125 MB/s × 0.1 s = **12.5 MB**.
- A TCP connection’s throughput is bounded by **window ÷ RTT**. With a 64 KB window and 100 ms RTT: 64 KB ÷ 0.1 s ≈ **640 KB/s ≈ 5 Mbps** — on a 1 Gbps link! Modern stacks use window scaling and tuned buffers, but loss and small windows still cap long-distance transfers.
- Practical lessons: use **multiple parallel connections** or HTTP/2-3 for long-distance bulk transfer, tune socket buffers, and move data **closer** (CDN/regional replicas) rather than adding bandwidth.

◀ QUICK RECALL

- System optimisation centres on **cutting latency bottlenecks** (fewer round trips, closer data) to **maximise throughput** across the cluster.

3.13 Network devices & firewalls

Device	Layer	Does what
Hub / repeater	1	Repeats bits to all ports (obsolete)
Switch	2	Learns MAC → port table; forwards frames within a LAN/VLAN
Router	3	Connects different networks; forwards packets by IP using routing tables; often does NAT + DHCP at home
Firewall	3-7	Allows/blocks traffic by IP, port, protocol (next-gen: app/user aware)
Load balancer	4 or 7	Spreads traffic over many servers (Chapter 7)
NAT / Internet gateway	3-4	Translates addresses; gives a private network a path to the Internet

◆ SENIOR INSIGHT — Stateless vs stateful filtering (very common cloud question)

- **Stateful** firewall (AWS Security Groups): remembers connections, so if outbound request is allowed, the reply is automatically allowed. Rules are allow-only.
- **Stateless** filter (AWS Network ACLs): evaluates every packet independently, so you must allow both directions explicitly (including ephemeral ports for replies). Supports allow and deny, evaluated in order.
- Best practice: **least privilege** — databases accept traffic only from the app tier’s security group, never from 0.0.0.0/0.

3.14 Troubleshooting toolbox & what errors mean

Question	Tool	Example
Is the host reachable? Latency?	ping	ping 8.8.8.8 (ICMP may be blocked)
Where is the path slow or broken?	tracert / mtr (Windows: tracert)	tracert example.com
Does the name resolve? To what?	dig, nslookup	dig +short example.com
Is the port open / service listening?	nc -vz host 443, telnet; locally ss -tulpn, lsof -i :3000	ss -tan grep ESTAB
Full HTTP exchange and timings?	curl -v	see timing command below
Certificate / TLS problems?	openssl s_client -connect host:443 -servername host	check chain, expiry, protocol
What packets are actually on the wire?	tcpdump, Wireshark	tcpdump -i eth0 port 443
My IP, routes, interfaces?	ip addr, ip route, ifconfig, ipconfig	

Break a request into phases with curl:

```
curl -o /dev/null -s https://example.com -w \
  "dns:%{time_namelookup} tcp:%{time_connect} tls:%{time_appconnect} \
  ttfb:%{time_starttransfer} total:%{time_total}\n"
# dns = name lookup | tcp = after handshake | tls = after TLS | ttfb = first byte | total = done
```

Error meanings (Node.js error codes in brackets)

Symptom	What actually happened	Typical causes
Connection refused (ECONNREFUSED)	Host reachable, but nothing listens on that port; kernel replied with RST immediately	Service down, wrong port, bound to 127.0.0.1 only
Timeout / hangs (ETIMEDOUT)	No reply at all: packets silently dropped or lost	Firewall / security group, wrong route, host down, SYN backlog full, overloaded server
Connection reset (ECONNRESET)	Peer (or middlebox) sent RST on an established connection	App crash, LB/NAT idle timeout, proxy limits, TLS mismatch
Name not found (ENOTFOUND, NXDOMAIN)	DNS could not resolve the name	Typo, missing record, DNS outage, wrong resolver
Address in use (EADDRINUSE)	Local port already bound	Duplicate process, old server not stopped
Too many open files (EMFILE)	File-descriptor limit reached (sockets count!)	Connection leak, low ulimit
Cannot assign requested address (EADDRNOTAVAIL)	Out of ephemeral ports / TIME_WAIT exhaustion	Too many short outbound connections to one destination → pool/keep-alive

◆ SENIOR INSIGHT — Refused vs timeout is the fastest first diagnosis

- **Refused** = you reached the machine; the problem is the process/port. **Timeout** = you did not get an answer; look at network path, firewalls, routing, or an overloaded target. Always set **connect and request timeouts** in clients — default “infinite” timeouts cause cascading failures (Chapter 12).

3.15 What this means for system design

Network fact	Design consequence
Every round trip costs RTT (0.5 ms in DC, up to 150 ms across continents)	Batch calls, avoid chatty services/N+1 calls, co-locate dependent services, use caching
TCP/TLS setup costs 2–3 RTT; slow start	Connection pooling, keep-alive, HTTP/2–3, TLS session resumption, persistent DB pools
Distance sets a latency floor	CDN, multi-region deployment, geo-DNS, read replicas near users
Networks fail, drop and duplicate packets	Timeouts, retries with exponential backoff + jitter, idempotent operations, circuit breakers
DNS caches by TTL	Don't rely on DNS alone for failover; use health-checked load balancers; plan TTL changes
Ports/fds/ephemeral ports are finite	Raise limits, pool connections, scale out, watch NAT gateway port usage
Private networks + NAT/firewalls	Public/private subnets, least-privilege security groups, bastion/VPN for admin access

3.16 Practice questions (try before looking)

P1. How many addresses are in a /20 block? Which smallest prefix fits 500 devices?

Answer: /20 $\rightarrow 2^{12} = 4,096$ addresses (4,094 usable). 500 devices need $2^9 = 512 \Rightarrow /23$ (510 usable).

P2. RTT is 80 ms. A browser opens https:// to a new site (DNS uncached, TLS 1.3). How long until the first response byte, ignoring server time?

Answer: DNS 1 + TCP 1 + TLS 1 + HTTP 1 = 4 RTT = ≈ 320 ms. With a warm connection: ~ 80 ms.

P3. TCP window 64 KB, RTT 50 ms. Max throughput?

Answer: $64 \text{ KB} \div 0.05 \text{ s} \approx 1.3 \text{ MB/s} \approx 10.5 \text{ Mbps}$ regardless of link speed.

P4. A service behind one NAT IP makes many short calls to a single API IP:443. Roughly how many simultaneous connections can it hold?

Answer: Limited by ephemeral source ports for that destination: $\sim 28k$ (Linux default) to $\sim 64k$; plus TIME_WAIT delays reuse.

P5. Telnet to a host on port 5432 says 'connection refused' but 'ping' works. What does it tell you?

Answer: Host and network are fine; no process is listening on 5432 (DB down or bound elsewhere) — not a firewall issue (that would time out).

Interview Q&A (30-second answers)

Q1. MAC address vs IP address?

MAC identifies an interface on the local link (hop-by-hop, Layer 2). IP identifies a host/network for end-to-end routing (Layer 3). MAC changes at every hop; IP stays the same except at NAT.

Q2. Why does a TCP connection need a three-way handshake?

To confirm both sides can send and receive and to synchronise initial sequence numbers before data flows; also protects against old duplicate connection requests.

Q3. TCP vs UDP — when each?

TCP for reliable ordered delivery (web, DB, APIs). UDP for low latency / loss-tolerant / broadcast (DNS, VoIP, games, QUIC).

Q4. What is a socket, and how can one server port serve thousands of clients?

A socket is an OS endpoint (IP:port + state). Each connection is identified by the 5-tuple, so many clients share one server port; limits are fds, memory and ephemeral ports.

Q5. Explain NAT and its downsides.

Rewrites private source IP/port to a shared public IP and keeps a mapping. Downsides: port exhaustion, breaks end-to-end addressing, shared-IP rate limiting issues; not a security control.

Q6. How does DNS resolution work and what is TTL?

Client $\rightarrow$ recursive resolver $\rightarrow$ root $\rightarrow$ TLD $\rightarrow$ authoritative; answers are cached at every level for TTL seconds. Short TTL = fast change, more lookups; long TTL = the reverse.

Q7. What happens when you type a URL?

DNS $\rightarrow$ TCP handshake $\rightarrow$ TLS handshake $\rightarrow$ HTTP request through CDN/LB/app/DB $\rightarrow$ response $\rightarrow$ browser parses and loads sub-resources. ~ 4 RTTs cold.

Q8. Difference between latency, bandwidth and throughput?

Latency = delay per operation; bandwidth = max capacity; throughput = actual delivered rate (limited by latency, window sizes, loss and server capacity).

Q9. 'Connection refused' vs 'timeout'?

Refused = host reached, no listener (RST). Timeout = no reply: firewall, routing, dead or overloaded host.

Q10. Why is HTTP/3 faster, and what is QUIC?

QUIC (UDP-based) merges transport + TLS 1.3 handshake into 1 RTT, avoids TCP head-of-line blocking with independent streams, and supports connection migration.

Chapter 3 Cheat Sheet

⚡ QUICK RECALL — Revise this in 2 minutes

- **Layers:** Physical → Data link (MAC) → Network (IP) → Transport (TCP/UDP, ports) → Application (HTTP/DNS/SSH).
“Please Do Not Throw Sausage Pizza Away.”
- **Encapsulation:** data → segment → packet → frame → bits. **IP end-to-end, MAC hop-by-hop.**
- **CIDR:** addresses = $2^{(32-\text{prefix})}$; /24 = 256, /16 = 65,536. Private: 10/8, 172.16/12, 192.168/16.
- **NAT:** many private IPs → one public IP (SNAT/PAT); watch port exhaustion; not a firewall.
- **Ports:** 0–65535; 80/443 web, 22 SSH, 53 DNS, 5432 Postgres, 3306 MySQL, 27017 Mongo, 6379 Redis, 9092 Kafka.
- **Socket** = IP:port endpoint (an fd). **5-tuple** = src IP, src port, dst IP, dst port, protocol.
- **TCP:** handshake (SYN, SYN-ACK, ACK) = 1 RTT; reliable via seq/ACK/retransmit; flow control (rwnd) + congestion control (cwnd); TIME_WAIT; head-of-line blocking.
- **UDP:** no handshake, no guarantees, low latency; DNS, VoIP, games; **QUIC/HTTP3** = reliable streams over UDP + TLS 1.3.
- **DNS:** resolver → root → TLD → authoritative; records A, AAAA, CNAME, NS, MX, TXT; TTL trades speed vs agility.
- **Cold HTTPS request** $\approx$ **DNS + TCP + TLS + HTTP = 4 RTT**. Reuse connections, use CDN, HTTP/2-3.
- **Throughput** $\leq$ **window** $\div$ **RTT**; BDP = bandwidth $\times$ RTT. Distance sets a latency floor ($\sim 5 \mu\text{s/km}$).
- **Refused** = reachable but no listener. **Timeout** = dropped/unreachable. **Reset** = peer/middlebox killed it.
- **Tools:** ping, traceroute, dig, curl -v, nc, ss, tcpdump, openssl s_client.